

Competitive Programming

Section 44 Editorial: String Algorithms II

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	4
1.1	Problem-specific proof map	4
1.2	Detailed solution guide	4
2	A. Trie Prefix Count	6
2.1	Restatement	6
2.2	Observations and algorithm	6
2.3	Correctness proof	6
2.4	Complexity	6
2.5	Full C++ solution	6
2.6	Full Python solution	6
2.7	Tests that matter	7
3	B. Palindrome Queries	8
3.1	Restatement	8
3.2	Observations and algorithm	8
3.3	Correctness proof	8
3.4	Complexity	8
3.5	Full C++ solution	8
3.6	Full Python solution	8
3.7	Tests that matter	9
4	C. Distinct Substrings	10
4.1	Restatement	10
4.2	Observations and algorithm	10
4.3	Correctness proof	10
4.4	Complexity	10
4.5	Full C++ solution	10
4.6	Full Python solution	10
4.7	Tests that matter	11
5	D. Unique Prefix Lengths	12
5.1	Restatement	12
5.2	Constraints and traps	12
5.3	Observations and trace	12
5.4	Algorithm	12
5.5	Correctness proof	13
5.6	Complexity	13
5.7	C++ implementation notes	13
5.8	Python implementation notes	13
5.9	Full C++ solution	13
5.10	Full Python solution	14
5.11	Common wrong answers	14
5.12	Tests that matter	14

6	What to carry forward	15
7	Testing discussion	16

1 Editorial overview

1.1 Problem-specific proof map

A. Trie Prefix Count. Incrementing every trie node visited by an inserted word makes that node's count equal to the number of words sharing its prefix. Following a query prefix therefore returns the desired count in its length.

B. Palindrome Queries. The local constraints permit direct mirrored comparison: a substring is a palindrome exactly when every symmetric pair is equal. This is $O(\text{length})$ per query and avoids probabilistic hash collisions.

C. Distinct Substrings. The local task uses a suffix automaton. State v contributes $\text{len}[v] - \text{len}[\text{link}[v]]$ distinct substring lengths, so summing these over non-root states is linear in the number of automaton states. Enumeration of all substrings is only a small-input stress-testing oracle.

1.2 Detailed solution guide

A. Trie Prefix Count. Each trie node represents exactly one prefix: the string on the path from the root to that node. During insertion, increment cnt at the root and after moving to each next node. Therefore, after all insertions, $\text{cnt}[v]$ is the number of dictionary words whose path passes through v , which is precisely the number of words with that prefix. A query walks the prefix characters; a missing edge means zero matches.

B. Palindrome Queries. For the local constraints, comparing mirrored characters is the clearest exact solution. For substring $[l, r]$, compare $s[l]$ with $s[r]$, then $s[l+1]$ with $s[r-1]$, and so on. If all pairs match, the substring equals its reverse. If one pair differs, the two strings differ at a mirrored position, so it is not a palindrome. In a larger contest version, the same condition can be checked with forward and reverse rolling hashes.

C. Distinct Substrings. The local solution is a suffix automaton. Each state contributes $\text{len}[v] - \text{len}[\text{link}[v]]$ new substrings: these are exactly the lengths whose end-position class first appears at that state. Summing over non-root states gives the answer. Enumerating every substring into a set is correct only for tiny random tests used to validate the automaton.

Common pitfalls. Trie counters count prefixes, not terminal words only. Palindrome queries must convert input indices consistently. For distinct substrings, enumeration is pedagogical and exact at small limits, but a suffix automaton is required for large external tasks.

This section shows three string representations.

- A uses a trie. Each inserted word increments counters on the nodes along its path, so the counter at the prefix endpoint is exactly the number of words with that prefix.
- B checks palindromes by comparing mirrored characters. For larger limits this can be replaced by forward/reverse hashes, but the local proof is exact.

- C counts distinct substrings with a suffix automaton. The local invariant is the contribution `len[v] - len[link[v]]` per non-root state; enumeration is reserved for tiny brute-force tests.

The editorial code uses the same asymptotically appropriate representation as the local contract; brute force appears only as a testing oracle.

2 A. Trie Prefix Count

2.1 Restatement

Given dictionary words and prefix queries, count words having each prefix.

2.2 Observations and algorithm

Each trie node is one prefix. Increment its counter for every inserted word visiting it; walk a query to its endpoint and return that counter, or zero on a missing edge. Inserting **apple**, **ape**, **bat** makes query **ap** return two.

2.3 Correctness proof

After every insertion, a node counter equals the words with that node's prefix, so the query endpoint is exactly the requested count. QED.

2.4 Complexity

Work is linear in total inserted and queried characters; memory is one node per distinct prefix.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct Node{array<int,26> nx{}; int cnt=0;};
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,q;if(!(cin>>n>>q)) return 0; v
```

2.6 Full Python solution

```
import sys
def main():
    lines=sys.stdin.read().splitlines(); n,q=map(int,lines[0].split()); nxt=[]; cnt=[]
    nxt.append({}); cnt.append(0)
    for w in lines[1:1+n]:
        v=0; cnt[v]+=1
        for ch in w:
```

```

        if ch not in nxt[v]: nxt[v][ch]=len(nxt); nxt.append({}); cnt.append(0)
        v=nxt[v][ch]; cnt[v]+=1
    out=[]
    for p in lines[1+n:1+n+q]:
        v=0; ok=True
        for ch in p:
            if ch not in nxt[v]: ok=False; break
            v=nxt[v][ch]
        out.append(str(cnt[v] if ok else 0))
    print("\n".join(out))
if __name__=="__main__": main()

```

2.7 Tests that matter

Test duplicate words, one word that prefixes another, an absent prefix, and a one-character query.

3 B. Palindrome Queries

3.1 Restatement

For each substring query, print whether the substring is a palindrome.

3.2 Observations and algorithm

The local bounds permit exact mirrored comparison. Convert inclusive 1-based $[l, r]$, compare outward characters, and stop on a mismatch. In `abacaba`, $[2, 4]$ is `bac` (NO) while $[3, 5]$ is `aca` (YES).

3.3 Correctness proof

A substring equals its reverse iff each mirrored character pair is equal, which is exactly the two-pointer condition. QED.

3.4 Complexity

Each query is $O(r-l+1)$ time and $O(1)$ extra memory; this exact approach is only suitable for the local small-query contract.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(C
```

3.6 Full Python solution

```
import sys
def main():
    s=sys.stdin.readline().strip(); q=int(sys.stdin.readline()); out=[]
    for _ in range(q):
        l,r=map(int,sys.stdin.readline().split()); sub=s[l-1:r]; out.append("YES" if sub==su
    print("\n".join(out))
if __name__=="__main__": main()
```

3.7 Tests that matter

Test singleton, full string, even and odd lengths, and a mismatch in the outer pair.

4 C. Distinct Substrings

4.1 Restatement

Print the number of distinct non-empty substrings of a string.

4.2 Observations and algorithm

Build a suffix automaton. State v contributes $\text{len}[v] - \text{len}[\text{link}[v]]$ new substring lengths; sum this over non-root states. For `ababa`, the answer is 9.

4.3 Correctness proof

Suffix links partition represented lengths into disjoint intervals, so every distinct nonempty substring belongs to exactly one state's contribution. Clones restore this invariant when a transition needs a shorter maximum length. QED.

4.4 Complexity

There are at most $2 \cdot n - 1$ states. The fixed-alphabet version is $O(n)$; the map-based listing is $O(n \log \text{alphabet})$.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct St{map<char,int> nx; int link=-1,len=0;};
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(cin>>s)) return 0; vec
```

4.6 Full Python solution

```
import sys
def main():
    s=sys.stdin.readline().strip(); next=[]; link=[]; length=[]
    next.append({}); link.append(-1); length.append(0); last=0
    for ch in s:
        cur=len(next); next.append({}); length.append(length[last]+1); link.append(0); p=las
```

```

while p!=-1 and ch not in next[p]: next[p][ch]=cur; p=link[p]
if p==-1: link[cur]=0
else:
    q=next[p][ch]
    if length[p]+1==length[q]: link[cur]=q
    else:
        clone=len(next); next.append(next[q].copy()); length.append(length[p]+1); li
        while p!=-1 and next[p].get(ch)==q: next[p][ch]=clone; p=link[p]
        link[q]=link[cur]=clone
    last=cur
print(sum(length[v]-length[link[v]] for v in range(1,len(next))))
if __name__=="__main__": main()

```

4.7 Tests that matter

Test a, aaa, all distinct characters, and an alternating string.

5 D. Unique Prefix Lengths

5.1 Restatement

For every input word, find the shortest nonempty prefix shared by exactly one input occurrence. Equal words are still separate occurrences, so no prefix can distinguish one copy from another. A word that is a prefix of a longer word may also have no answer.

This is the blind trie exercise for the section: the task asks for a property of all prefixes of each word, not merely a count for one supplied prefix.

5.2 Constraints and traps

There are up to 200,000 words and the same bound on total characters. Testing every prefix against every word is quadratic. The answer is an input-order answer, so duplicate words need repeated -1 lines rather than being merged.

Do not mistake “the word ends here” for “the prefix is unique.” For **do** and **dog**, the prefix **do** is a prefix of both input words even though only one word ends at that trie node.

5.3 Observations and trace

In a trie, store `pass[node]`: the number of input words whose path visits the node. During insertion, increment it after every character. Then a prefix is unique exactly when the node reached by that prefix has `pass == 1`.

Insert the sample’s **cat**, **car**, **dog**, **do**, **cat**. The node for **ca** has `pass=3` because it serves both copies of **cat** and **car**. The node for **car** has `pass=1`, so **car** needs length three. The node for **do** has `pass=2`, and there is no later node for the shorter word **do**, so its answer is -1.

5.4 Algorithm

```
make a trie whose nodes store child links and pass counts
for every word:
    walk/create its character path
    increment pass at each reached node

for every word in input order:
    walk its trie path from the root
    print the first position whose node has pass = 1
    print -1 if no such position occurs
```

5.5 Correctness proof

Lemma. For every trie node representing prefix p , $\text{pass}[\text{node}]$ equals the number of input words beginning with p .

Proof. A word visits exactly the nodes for its nonempty prefixes during insertion. The algorithm increments this node once for that word and never for a word that does not have the prefix. Summing over all input occurrences gives the stated count. $\square$

Theorem. The algorithm prints the required shortest unique-prefix length for every word.

Proof. By the lemma, a scanned prefix is unique exactly when its node has $\text{pass}=1$. The second traversal checks prefixes in increasing length order, so the first such node is the shortest valid prefix. If none is found, every nonempty prefix is shared by at least two occurrences and -1 is correct. $\square$

5.6 Complexity

Let L be the total input length.

```
Time:    $O(L)$ 
Memory:  $O(L)$ 
```

5.7 C++ implementation notes

An `array<int,26>` gives fixed lowercase transitions. A node index of zero is the root; newly allocated children receive positive indices. Increment only the destination node's `pass` count, because the empty prefix is not allowed.

5.8 Python implementation notes

A list of dictionaries keeps the implementation direct. Use byte input for large files if desired; the reference decodes after splitting because input is restricted to lowercase ASCII.

5.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
struct Node { array<int,26> next{}; int pass = 0; };
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; if (!(cin >> n)) return 0; vector<string> words(n); vector<Node> trie(1);
    for (string &word : words) { cin >> word; int node = 0; for (char ch : word) { int c=ch-'a';
        for (const string &word : words) { int node=0, answer=-1; for (int i=0;i<(int)word.size()
```

5.10 Full Python solution

```
import sys

def main() -> None:
    words = sys.stdin.buffer.read().split()
    if not words: return
    n = int(words[0]); words = [word.decode() for word in words[1:n + 1]]
    children = [{}]; passed = [0]
    for word in words:
        node = 0
        for char in word:
            if char not in children[node]:
                children[node][char] = len(children); children.append({}); passed.append(0)
            node = children[node][char]; passed[node] += 1
    output = []
    for word in words:
        node = 0; answer = -1
        for length, char in enumerate(word, 1):
            node = children[node][char]
            if passed[node] == 1: answer = length; break
        output.append(str(answer))
    sys.stdout.write("\n".join(output) + "\n")

if __name__ == "__main__": main()
```

5.11 Common wrong answers

Counting only words ending at a node instead of every word passing through it.
Deduplicating equal strings before answering.
Returning a prefix for do when dog is also present.
Using the root count as an empty-prefix answer.
Scanning all words for every candidate prefix.

5.12 Tests that matter

Identical words: every copy must print -1.
A word that is a prefix of another word.
Words sharing a long prefix before branching.
A one-word input: answer is 1 for a nonempty word.
200,000 identical one-letter words: checks linear memory and output.

6 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.

7 Testing discussion

Brute-force small trie prefixes by `word.startswith(prefix)`, palindrome queries by reverse comparison, and distinct substrings with a set of all slices. Then run:

```
CP_TARGET=solution python3 sections/44_string_algorithms_ii/check.py
```